

CAS 732 Replication Project: A Quantitative Evaluation of Self-Attention for Neural Machine Translation

Julian Cecchini

McMaster University

cecchinj@mcmaster.ca

Abstract

The Transformer architecture demonstrated that neural machine translation can achieve state-of-the-art performance using self-attention alone, without recurrence or convolution. Understanding this architecture is fundamental to modern language models. This project presents a faithful replication of the Transformer base model introduced by Vaswani et al. (2017), with the objective of reproducing its reported BLEU performance on the WMT14 English–German translation task. A complete implementation was constructed in Python, including custom preprocessing, batching, and evaluation pipelines to match the original experimental conditions. The replicated model achieves a BLEU score of 24.6, corresponding to a deviation of 9.89% from the original reported score of 27.3. These results indicate that the performance of the original Transformer can be reproduced within a reasonable margin when critical implementation details are carefully reconstructed.

1 Introduction

Large language models (LLMs), such as ChatGPT (OpenAI et al., 2023) and DeepSeek (DeepSeek-AI et al., 2024), have gained widespread adoption due to their strong performance across a variety of natural language processing (NLP) tasks. These models are fundamentally built upon the Transformer architecture introduced by Vaswani et al. (2017).

The Transformer architecture initially came to prominence through the task of neural machine translation (NMT). In this setting, a model is expected to translate from a source language to a target language while preserving semantic meaning and generating fluent output.

More formally, treating words as tokens, NMT models are typically trained under a next-token prediction objective. Given a source sequence

$x = (x_1, \dots, x_n)$ and a target sequence $y = (y_1, \dots, y_m)$, the model learns to factorize the conditional distribution as

$$P(y \mid x) = \prod_{t=1}^m P(y_t \mid y_{<t}, x),$$

where $y_{<t} = (y_1, \dots, y_{t-1})$ denotes the previously generated tokens. The goal of training is maximizing the log-likelihood of the target sequence under this factorization, and one way of accomplishing this is minimizing the Kullback–Leibler divergence (KL-loss) between smoothed target distributions and model distributions.

Model inference generates translations autoregressively: it takes tokens generated for a sequence so far as a condition for its learned distributions, selects the next individual token based on those distributions and adds it to the sequence, and continues until a full translation is formed.

Prior to the Transformer, NMT systems were predominantly based on recurrent neural networks (RNNs) (Sutskever et al., 2014; Bahdanau et al., 2015) or convolutional neural networks (CNNs) (Gehring et al., 2017). While these models significantly outperformed earlier phrase-based statistical machine translation systems (Koehn et al., 2003), they were constrained by their ability to model long-range dependencies efficiently. RNNs require sequential computation from one token to the next, limiting parallelization, while CNNs rely on fixed-size kernels that require deep stacking to capture broader context. In both cases, tokens that are far apart in a sequence may have their contextual relationships weakened.

Vaswani et al. (2017) challenged the necessity of recurrence and convolution through the introduction of self-attention as the primary mechanism for sequence modeling. Self-attention enables direct interactions between all tokens in a sequence, allowing the model to capture long-range

dependencies while remaining fully parallelizable. Using this approach, the Transformer achieved state-of-the-art performance on the WMT 2014 English–German (EN-DE) translation benchmark.

Performance in this task is measured using BLEU (Papineni et al., 2002), which evaluates translation quality by computing n-gram overlap between predicted and reference translations while incorporating a brevity penalty. The strong BLEU performance of the Transformer demonstrated that self-attention alone was sufficient for high-quality machine translation, marking a shift away from recurrent and convolutional approaches.

The objective of this project is to reproduce the results of Vaswani et al. (2017) by implementing the Transformer base model and establish claims of its performance. Specifically, this work seeks to verify the following hypothesis:

Claim: Using the Transformer *base* configuration from Vaswani et al. (2017) that is built exclusively on self-attention mechanisms can achieve the reported BLEU score of 27.3 within $\pm 10\%$ error on the WMT 2014 EN-DE dataset, putting its efficacy on the order of contemporary RNNs and CNNs.

To accomplish this, the model and its corresponding pipeline for data processing, batching, and evaluation is implemented in Python with major reliance on PyTorch. The full implementation is available on GitHub¹. The remainder of this report details the dataset, methodology, experimental results, and reflections obtained from this replication effort.

2 Related Work

2.1 Model Architecture

The primary reference for this work is naturally the paper that it seeks to replicate, Vaswani et al. (2017), which introduced the Transformer architecture and demonstrated state-of-the-art performance on NMT tasks.

While the architectural contributions of this work are well-documented, it is not very prescriptive for an implementation in itself. This lack of specificity combined with no associated github repository has made faithful replication a non-trivial task. Fortunately, this replication is not the first of its kind and there are others like it to reference. Rush (2018) is one such reference, pro-

viding end-to-end reference code for the model as well as the training pipeline. This paper builds faithfully off the guidance of it, largely borrowing from its example code.

2.2 Tokenization

Tokenization is a process where a piece of text is split into statistically significant subwords according to a vocabulary constructed from a reference corpus. NMT models from the late 2010s up until today have all utilized tokenization during training to help establish root word relations across languages. These relations allow models to more accurately stem from root words into complex words across different contexts. A popular tokenization method still used today is known as BPE-tokenization from Sennrich et al. (2016). It is this same method that was utilized by Vaswani et al. (2017) although the specific nature of their subword boundary handling was not described. Therefore, this work decided to closely emulate the original BPE-tokenization method in hopes it was not deviated from significantly for the original Transformer.

2.3 BLEU benchmark

Evaluation of translation quality by NMT models is performed using BLEU (Papineni et al., 2002), which measures n-gram overlap between predicted and reference translations while applying a brevity penalty. Although the BLEU formulation itself is mathematically well-defined, its practical computation depends heavily on preprocessing decisions such as tokenization and normalization. This sensitivity is extensively discussed by Post (2018), who demonstrate that seemingly minor differences in tokenization pipelines can lead to significant variations in BLEU scores. This resulted in the creation of modern standardized tools such as SacreBLEU from Post (2018), that enforce consistent preprocessing for modern NMT assessment. Tools like this did not exist for the original Transformer evaluation, so this work instead implements a bespoke BLEU pipeline intended to emulate historical practices. The wisdom from Post (2018) that tokenization is immensely relevant to the BLEU score influenced the construction of this pipeline. Namely, it inspired the inclusion of BPE tokenization for training and Moses-style tokenization from Koehn et al. (2007) on model predictions right before evaluation.

¹<https://github.com/N4T10N01/AttentionIsWhatINeed>

3 Dataset

The experiments in this work are conducted on the WMT14 EN-DE dataset, a widely used benchmark in neural machine translation introduced by the Workshop on Machine Translation (WMT). The dataset consists of approximately 4.5 million sentence pairs in the training split, with separate validation and test sets of 3,000 sentence pairs each. The corpus is composed primarily of news-domain text, making it well-suited for evaluating general-purpose translation models.

A defining characteristic of this dataset is the high variability in sentence length. Sentences range from very short phrases to long, complex structures, resulting in a highly non-uniform distribution of sequence lengths. This variability introduces several challenges for training Transformer models, particularly with respect to batching and computational efficiency.

In standard tensor-based training pipelines, all sequences within a batch must be padded to the same length. Specifically, each sequence is extended to match the length of the longest sequence in the batch. When there is a large disparity between the shortest and longest sequences in a batch, this results in excessive padding. From a computational standpoint, this is inefficient, as a significant portion of the computation is performed on padding tokens that carry no meaningful information. From a learning perspective, this can also degrade performance, as the effective signal-to-noise ratio of each batch is reduced.

To mitigate this issue, the original Transformer paper takes efforts to organize the batches such that sequences of similar lengths are grouped together. This reduces the amount of padding required and improves both computational efficiency and training stability.

In addition to padding issues, the sentence length variability presents another challenge related to the number of tokens processed per batch. When batching is performed purely based on sentence count, even if length is controlled within a batch so padding is minimized, one batch can have an average sentence length much longer than another. This means the total number of tokens per batch can fluctuate significantly between the batches if the sentence count is kept identical between them. This leads to inconsistent gradient magnitudes across training steps, which can negatively impact optimization.

To address this, the original paper by Vaswani et al. (2017) introduces the concept of a token-based batch size, specifying that each batch should contain approximately 25,000 tokens. This ensures that each optimization step processes a roughly constant amount of data, leading to more stable gradient updates.

These concerns of padding and token count within a batch that arise from the highly variable sentence length of the WMT14 dataset in a NMT context, are not particularly discussed in the original paper, but they are potentially critical to achieving not only a faithful and effective replication, but a quality NMT model in general.

The dataset can be currently accessed through modern dataset libraries, like datasets from PyTorch, that interface with publicly hosted repositories, like HuggingFace². It is provided in pre-defined splits corresponding to the training, validation, and test sets as originally procured from WMT.

4 Methodology

4.1 Model Implementation

The Transformer architecture is implemented in PyTorch following the original design according to the tutorial given by Rush (2018). All major components are constructed manually, including multi-head attention, feedforward layers, positional encodings, and residual connections.

A key implementation insight involves the multi-head attention projection. Although the paper describes projections into lower-dimensional subspaces, the implementation uses full-dimensional linear layers that are reshaped to produce multiple attention heads. This effectively learns a combined projection matrix that captures all heads simultaneously.

Positional encodings are implemented using sinusoidal functions, where each dimension corresponds to a sinusoid of a different frequency. Lower-frequency components encode global position, while higher-frequency components capture local relationships. This design ensures that positional information is both unique and linearly recoverable. For easy reference, a copy of the original Transformer architecture and its correspondence to the implementation used is found in Figure 1.

²<https://huggingface.co/datasets/wmt/wmt14/viewer/de-en>

```

def make_model(src_vocab, tgt_vocab, N=6,
              d_model=512, d_ff=2048, h=8, dropout=0.1):
    "Helper: Construct a model from hyperparameters."
    c = copy.deepcopy
    attn = MultiHeadedAttention(h, d_model)
    ff = PositionwiseFeedForward(d_model, d_ff, dropout)
    position = PositionalEncoding(d_model, dropout)
    model = EncoderDecoder(
        Encoder(EncoderLayer(d_model, c(attn), c(ff), dropout), N),
        Decoder(DecoderLayer(d_model, c(attn), c(attn),
                              c(ff), dropout), N),
        nn.Sequential(Embeddings(d_model, src_vocab), c(position)),
        nn.Sequential(Embeddings(d_model, tgt_vocab), c(position)),
        Generator(d_model, tgt_vocab))

    # This was important from their code.
    # Initialize parameters with Glorot / fan_avg.
    for p in model.parameters():
        if p.dim() > 1:
            nn.init.xavier_uniform(p)
    return model

```

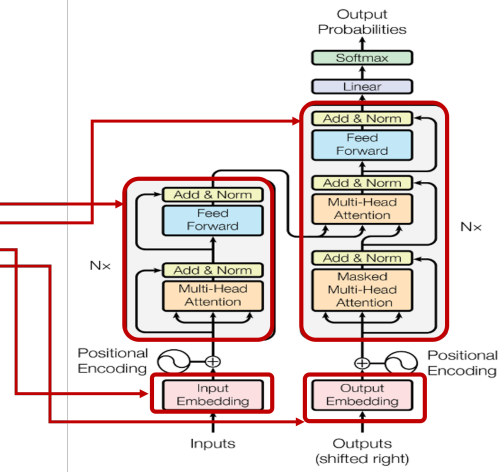


Figure 1: The Transformer - model architecture.

Figure 1: Transformer architecture implementation used in the project.

4.2 Tokenization Implementation

The original Transformer paper specifies the use of a shared source-target vocabulary of approximately 37,000 tokens for a BPE tokenizer, but does not detail the exact pre-processing steps used to construct it, like how word boundary tokens are added.

Thus, a custom BPE tokenizer is implemented from scratch based on the original implementation given by [Sennrich et al. \(2016\)](#). This tokenizer is also jointly trained on the English and German components of the WMT14 training splits to construct a vocabulary totalling 37,000 tokens as the original paper. This is likely effective due to the shared alphabet and subword units between English and German.

Word boundary markers of (`</w>`) were added to tokens that represented the end of words to ensure correct reconstruction of sentences during evaluation. This decision is critical for accurate BLEU scoring, as incorrect token boundaries can significantly distort n-gram matching. An alternative to this that the original paper may have used would be adding boundary markers like (`@@`) that mark intermediate parts of words.

4.3 Batch Construction

This replication effort constructs batches by sentence count. The original paper used token count but did not prescribe how it was accomplished (i.e., what tolerance there was for sentence length or if sentences were truncated or the maximum sentence length permitted). Therefore, for benefit of simplicity and in part due to schedule con-

straints, sentence count was used.

At earlier points in experimentation, batches had a cap on how many tokens they could contain. This is referred to as token capping and was enforced by counting how many tokens existed in a batch after construction and removing sentences until a specified threshold was met. The main reason for this was to avoid out-of-memory (OOM) issues that occurred for larger batches, but it also prevented batches from exceeding the 25000 token count per batch of the original paper. However, it was later discovered the token cap could be removed without OOM issues reoccurring and, thus, was removed.

Finally, for reasons highlighted later on, matching sentence lengths within batches was added into the pipeline eventually. It was accomplished by first sorting the corpus by sentence length (max between source and target), then randomly shuffling over equally sized pools of this sorted corpus, and finally randomizing the order of those pools before use across epochs to prevent bias from deterministically ordered sentences. Herein, this is referred to as quantile partitioning.

4.4 Parallelization of Training

The creation of batches as well as gradient calculation across batches had to be parallelized across processes to obtain reasonable completion times for training. The training pipeline code initially borrowed from [Rush \(2018\)](#) did not cut it due to its CUDA usage that was out of date for GPUs from 2022 or later. The pipeline was therefore updated to use the Distributed Data Parallel (DDP) module from Pytorch to achieve parallelization.

In this code, training is split across individual DDP processes (according to a parameter specified). Each DDP process is responsible for gathering a batch according to its own DistributedSampler object, calculating the gradient loss for this batch, and synchronizing its loss with other processes before moving onto another batch.

This means with n processes each working a X sentence-long batch:

$$\text{global batch size} = nX$$

For example, below a process batch size of 512 is used with 4 processes, creating a global batch size of $4 \times (512) = 2048$ sentences. Batch sizes in the below sections will be given solely in per process form.

4.5 Hardware Setup

Training was done exclusively on the FIR cluster of the Digital Research Alliance of Canada. FIR is a high-performance computing (HPC) cluster with 165K CPU cores, 640 H100 GPUs, 50 PB of storage. Only 4 GPUs were available by request at a time for training. Thus, for every single training cycle described in this work, exactly 4 processes were used. A training batch of 20 epochs averaged around 4.5 hours roughly.

4.6 Model Hyperparameters

Per the sage advice of [Rush \(2018\)](#), majority of hyperparameters were copied over from [Vaswani et al. \(2017\)](#) for the base configuration. These can be found in Table 1.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}
base	6	512	2048	8	64	64	0.1	0.1
Optimizer: Adam ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$)								

Table 1: Model hyperparameters (base configuration) and optimizer settings.

N corresponds to the number of complete encoder-decoder layers stacked onto one another, d_{model} corresponds to the size of embedding vectors, d_{ff} is the size of the feedforward layers in the encoder-decoder stack, h is number of heads contained within a multi-head layer, d_k and d_v are the dimensions of the projected query and key, and value vector dimensions respectively in scaled-dot attention unit of each head, P_{drop} is the dropout rate of neurons across layers during training, and

ϵ_{ls} is the amount of smoothing applied to target distributions to change them from singular masses.

The sole hyperparameters which were not identically matched were batch size, steps per epoch, and optimizer warm-up threshold. This is due to this work utilizing the aforementioned sentence-based batching strategy instead of token-based. For example, if batch A contains 8 sentences each 10 words long and batch B contains 8 sentences each 11 words long, the token count shared between them will vary. Under this regime, however, the amount of sentences per batch necessary to approximate the token-based strategy as the original paper can still be calculated.

Let T denote the target token count per batch and $\bar{L}$ denote the average number of tokens per sentence. Then the relation between the two batching regimes can be stated as:

$$\text{sences per batch} \approx \frac{T}{\bar{L}}$$

It was calculated from the train split of the WMT14 dataset had roughly 139 million tokens in total with the custom BPE tokenizer created and with 4.5 million sentences, this comes to roughly 30 tokens per sentence on average. So with a goal of 25000 tokens per batch, $25000/30 = 833$ sentences per batch should suffice.

Unfortunately, maintaining a sentence-based regime for batching introduces a tradeoff between matching token counts and matching the number of optimizer steps taken. The original paper reports 100,000 training steps over 10 epoch, or 10,000 training steps per epoch with a optimizer warm-up threshold of 4000. 833 sentences per batch across 4.5 million sentences of the training set results in roughly 5400 steps per epoch, half as many as the original paper. To match the step count per epoch of 10,000 from the original paper across 4.5 million sentences, 450 sentences are required per epoch. Of course 450 sentences translates to only 13,500 tokens on average. As a result of this trade-off, batch sizes between 450 and 833 sentences are explored for batch size.

In the same vein, warm-up thresholds for the optimizer are explored between 2000 and 4500. A lower warm-up threshold means the learning rate peaks higher and starts decaying faster, which makes sense for lower step counts per epoch, and vice versa with higher thresholds for epochs with higher step counts.

4.7 Evaluation

BLEU scoring is implemented manually to emulate historical evaluation pipelines. This means Moses-style tokenization from Koehn et al. (2007) is approximated and applied to predicted translations before they are scored, as it was likely used in the original work.

The simplest way to have the model generate translations for evaluation is greedy decoding, where the token distribution generated by the model is inspected at each autoregressive step and the token with the highest probability is selected to chain the sequence. A more complex form is beam search decoding where multiple token selections are inspected at each step and the sequence with the highest probability is selected. This improves translation quality by exploring multiple candidate sequences at the cost of higher compute complexity, and is the method the original paper used.

The higher compute complexity is non-trivial and practically takes 2 hours more to evaluate locally compared to greedy decoding which takes 1.5 hours. HPC resources could not be spared for this calculation. Therefore, it was only applied at the end of the project schedule to the best models obtained. Beam search parameters used were identical to the original paper, with a beam width of 4, $\alpha = 0.6$ for length penalty, and a termination length of 50 tokens.

5 Results and Evaluation

The results of this work are placed in context of an evolving system, where multiple implementation issues were identified and resolved progressively. This is done in spirit of the replication work’s goal to educate. As such, performance improvements are not solely attributable to hyperparameter tuning, but also to the correction of critical bugs and reconstruction of missing pipeline components.

5.1 Initial Obstacle: Sequence Non-Termination

The first model successfully produced used a batch size of 512 sentences per process and a warm-up threshold of 2000 with an otherwise faithful configuration. After 10 epochs of training, the model produced the following German-to-English (DE-EN) sample translation:

Original: Hallo, wie geht es dir?
Translation: Hallo , how are you???,
Hallo, Hallo, how are you???, ...

The generated output exhibits characteristics that hinted the right path was being traversed. Mainly, local translation quality was partially preserved, as evidenced by the correct rendering of phrases such as “how are you.” However, the sequence fails to terminate and instead continues generating repetitive fragments. This indicated that while the model learned token-level and short-range dependencies, it had not learned the concept of sequence completion.

Before this bug was resolved, another model was produced using 128 sentences per process and warm-up threshold of 4000. The KL-loss over the course of each model’s training can be found in Figure 2, with final values around 2.5 and 1.8 respectively. The new model continuously outperforming the first one across training signified the hyperparameter changes made were favourable. This was sensible as they aligned more with selections of the original paper. Therefore, a lower batch size and higher warm-up rate was maintained going forward.

5.2 Impact of Bug Fix

Further investigation revealed that models’ inability to terminate sequences was not due to deficiencies in the model architecture or optimization process, but rather a subtle bug in the data preprocessing pipeline. This bug resulted in the omission of end-of-sequence (EOS) tokens from all target sequences. Consequently, the model was never exposed to proper sequence termination during training, and the probability of emitting an EOS token remained near zero during decoding.

After correcting the preprocessing pipeline to explicitly insert EOS tokens, immediate improvements were observed. A translation from an early epoch of the next model with 200 sentences per process and 4500 warm-up steps following this fix is shown below:

Original: Hallo, wie geht es dir?
Translation: Hello, how do you?

While still imperfect, this output demonstrated that the model had begun to learn both sentence structure and termination.

It is important to note up until this point, there were several inconsistencies in the experimental setup with respect to the original paper of (Vaswani et al., 2017). One inconsistency was that all trained models performed translation in the reverse direction relative to the intended benchmark.

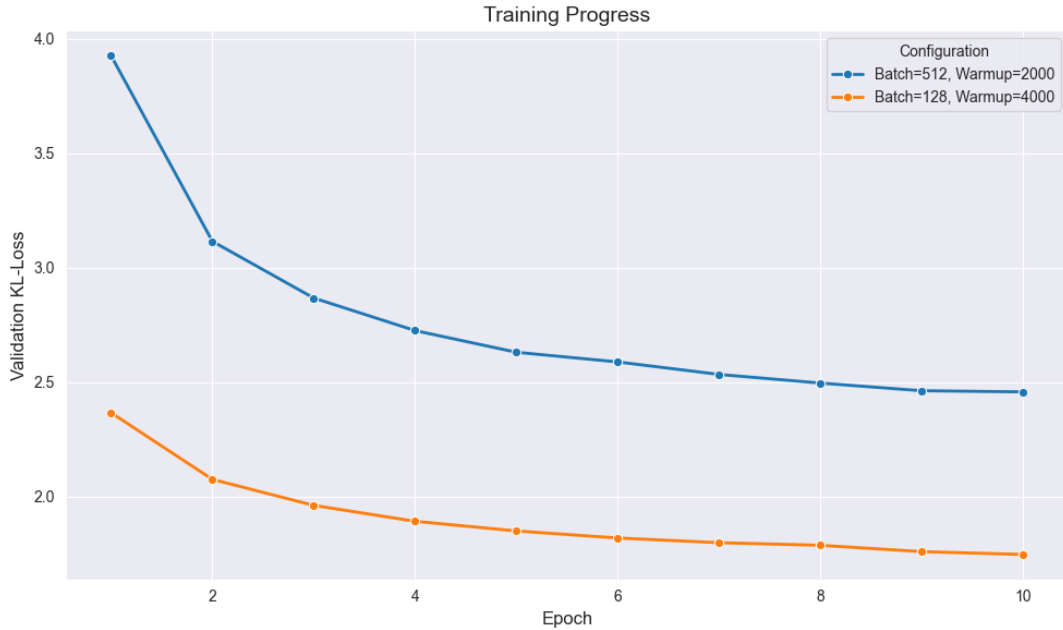


Figure 2: KL-loss over 10 epochs between initial models with no sequence termination

They translated from DE-EN instead of EN-DE. This behavior originated from an artifact of the legacy tutorial code, which defined the translation task differently than expected. Although this discrepancy was initially overlooked, several models had already been trained under this regime before it was identified.

Despite this, experimentation in the reversed translation direction was temporarily maintained even after the issue was discovered. This decision was pragmatic: it is much easier for researchers to substantiate translation quality in their native language. For researchers of this project, that was English. As a result, intermediate debugging and validation benefited from this setup, even though it did not align with the final evaluation protocol.

Following the bug fix, the clear improvement in translation quality suggested that the training pipeline had matured enough to produce worthwhile results in EN-DE translation. Consequently, the final four models were trained in the correct translation direction for the benchmark. These models were trained from scratch for 20 epochs to check for convergence and possible improvement over the initial 10 epochs suggested by the original paper. Their loss over time is shown in Figure 3.

These final models were initialized using hyperparameters consistent with the initial post-bug-fix configuration, after which further experimentation diverged beyond hyperparameter tuning. In par-

ticular, attention was shifted toward improving the other inconsistencies in the experimental design: inconsistent sentence lengths and token capping in batch construction.

5.3 Inconsistent Sentence Lengths

Early experiments used completely randomized batches, which resulted in higher variance in sentence lengths within each batch. As established, this leads to excessive padding, as all sequences must be padded to the maximum length in the batch. It was believed addressing this using quantile partitioning to ensure batches had less variance in their sentence lengths would be beneficial to training. Indeed, the next set of models produced utilized this pipeline change after the initial post-bug-fix model.

As seen by lines labeled **length matching** in Figure 3, introduction of this change did not appear to significantly change model quality. KL-loss remained relatively similar to the initial post-bug-fix model. Barring an erroneous implementation, this may indicate the shuffling pools used in quantile partitioning strategy were too large to negate padding more than a completely random batch sampler, or that the amount of padding avoided in the first place by the original paper was not significant.

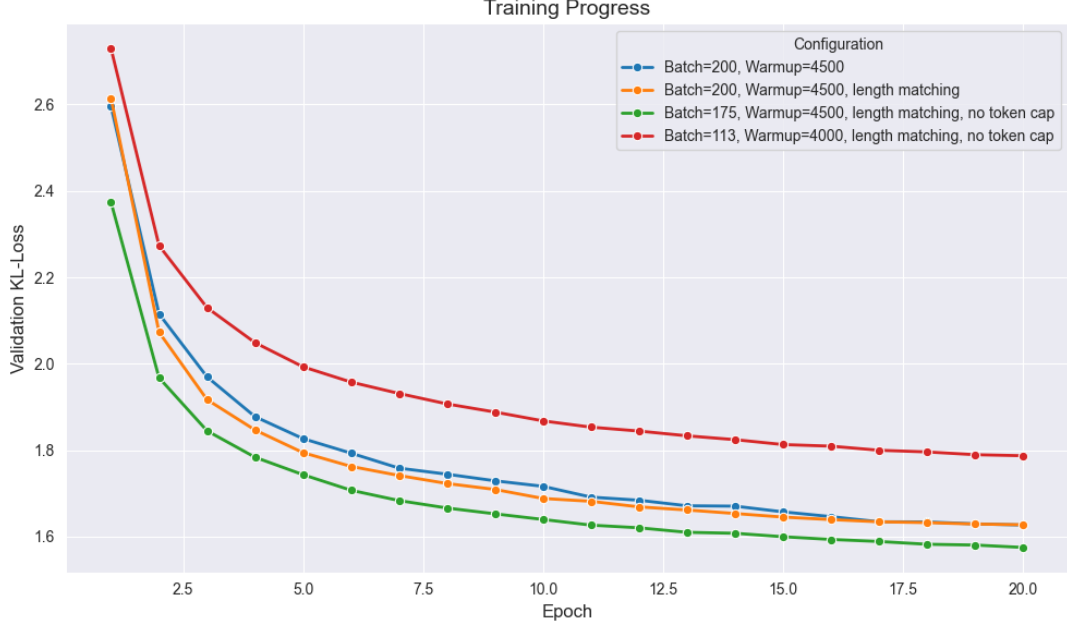


Figure 3: KL-loss over 20 epochs between final models with sequence termination

5.4 Token Capping

Early experiments also capped total tokens used per batch. This was done to avoid OOM issues for larger batches (400-500 sentences per process) but it was found it was not necessary for smaller batches (100 to 200 sentences per process). The final few models trained within the project had no token cap enforced upon their batches during training and are labeled with **no token cap** in Figure 3.

The first model created with this change produced the best KL-loss of any model in this project, settling around 1.57 by the 20th epoch. A lower batch size of 175 was assigned to this model to steer away from OOM issues as well as possibly keep the total tokens inspected per epoch similar to the previous model. To check whether the improvement was fully attributable to batch size change, a final model was made by reducing the batch size of this top-performing model to 113 sentences per process with a warm-up of 4000 steps. Surprisingly, this produced a large degradation in performance with a loss of 1.78, a level that had not been seen since before the bug fix. It should be noted also the choice of 113 sentences per processor with 4000 warm-up steps was intended to precisely capture the original paper’s steps per epoch and warm-up threshold, as this resulted in 113×4 processors = 452 sentences per global batch, so roughly 10,000 steps per epoch

with respect to the 4.5 million sentence size of WMT14’s training split. Therefore, the increase was not clearly attributable to batch size decreases or removing the token cap but the project schedule did not provide for more time to isolate the improvement.

5.5 Final Model Evaluations

Evaluation of the final models was conducted on the WMT14 EN-DE test set using the custom BLEU pipeline described earlier. The results are summarized in Table 2.

Configuration	Batch Size	warm-up	BLEU
Baseline	200	4500	23.1
+ Length	200	4500	22.9
+ No Tok Cap	175	4500	24.6
Paper Approx	113	4000	20.7

Table 2: BLEU score comparison across configurations.

The BLEU scores correspond closely with KL-losses found above. The lower the loss, the better the score. Although, this may not generally be case. The original paper found there were cases where perplexity, a metric neighbouring KL-loss, could worsen while BLEU score improved, like when smoothing was applied to target distributions.

5.6 Conclusion

This work set out to faithfully replicate the Transformer architecture introduced by Vaswani et al. (2017) and reproduce its reported BLEU score on the WMT14 EN-DE translation task. Through extensive reconstruction of the training pipeline—including tokenization, batching, and evaluation—a final BLEU score of 24.6 was achieved, corresponding to a deviation of 9.89% from the original reported result of 27.3. These findings support the central hypothesis that the Transformer’s performance is reproducible within a reasonable tolerance of 10%.

There are two big caveats to this result. One is that hyperparameters used were a bit different than the base configuration. The steps taken for the 20th and final epoch of the top-performing model totaled bit more than the 100,000 taken by the base configuration emulated. A batch size of 175 sentences per process results in approximately 6428 steps per epoch. Across 20 epochs, this is around 128,500 steps in total. It is dubious it had what could be considered a relevant advantage and still did so much worse. On the flip side, it would have only had around 21,000 tokens per batch assuming 30 tokens per sentence on average, which could prove a bigger detriment.

The second caveat is that a deviation of 2.7 on the BLEU benchmark is quite large, eclipsing the improvement Vaswani et al. (2017) claimed to have over other NMT models that were their competition at the time. Because replication successfully achieved a result within its goal tolerance and there were multiple areas where it may have deviated from the original paper due its omission of details, it cannot be said the exact scores of the original paper were exaggerated. Still, it can be said those scores require additional, specialized guidance or experience not provided by the original paper or tutorial used.

Ultimately, this study highlights the gap between theoretical descriptions and practical implementations in machine learning research.

6 Reflections

The replication of Vaswani et al. (2017) can be broadly decomposed into three primary components: (1) reconstructing the model architecture layer by layer, (2) collecting and preprocessing the dataset, and (3) selecting appropriate hyperparameters for training. The first component was rel-

atively straightforward. With the aid of the tutorial implementation by Rush (2018), the availability of PyTorch, and the architectural diagram provided in the original paper, the model could be systematically assembled and verified. Each component—including multi-head attention, positional encoding, and feed-forward layers—could be implemented and tested in isolation, making it possible to build confidence in correctness incrementally.

In contrast, data collection and preprocessing proved significantly more challenging. This is mainly because several dependencies for this phase relied upon by both the tutorial and the original codebase were deprecated or incompatible with modern environments. This was particularly evident in the case of the BPE tokenization, which had to be reimplemented from scratch due to the absence of maintained libraries consistent with the original methodology. Furthermore, neither the paper nor the tutorial provided explicit guidance on how batches should be constructed to minimize padding. While it was stated that sentences of similar lengths were grouped together, the exact mechanism was unspecified, requiring this aspect of the pipeline to be designed independently.

Hyperparameter selection, by comparison, was also relatively straightforward. The original paper provides clear guidance on key parameters such as optimizer parameters and model dimensions. These prescriptions were recommended by the tutorial and translated effectively into practice, providing a reliable starting point for experimentation. Deviations from these values were explored cautiously, but the baseline configuration described in the paper proved to be a strong and stable foundation.

At a lower level, the most technically demanding aspect of the project was enabling efficient multi-GPU training. This was essential for reducing training time and enabling iteration over multiple experimental configurations. Achieving this required upgrading the software stack (Python, PyTorch, and CUDA) multiple times to ensure compatibility with available hardware, first for a local machine equipped with an NVIDIA 5070 Ti GPU and later for the H100 GPUs available through Compute Canada. These upgrades necessitated significant refactoring of the codebase provided by Rush (2018), including replacing outdated data iterators with PyTorch DataLoader

constructs, incorporating DistributedSampler for dataset partitioning, and restructuring the training loop to operate under the DDP module of Pytorch. This process introduced numerous bugs and required substantial debugging effort, ultimately accounting for a significant portion of the total development time.

From a broader perspective, several key lessons emerged regarding the process of replicating research papers. First, while papers are effective for validating implementations, they are often insufficient as standalone guides for constructing them. Critical implementation details are frequently omitted or abstracted away, making it difficult to reproduce results without supplementary resources. In this project, the tutorial served as an essential bridge between the conceptual description in the paper and a working implementation. Without it, achieving a faithful replication would have been exceptionally more difficult.

Second, the approach to environment versioning was found to be suboptimal. The initial strategy prioritized using older software versions to match the original implementation as closely as possible, beginning with Python 3.6 and earlier PyTorch releases. This approach was intended to minimize discrepancies but ultimately introduced unnecessary complications. While minor compatibility issues arose when transitioning to newer versions of PyTorch, these were generally manageable, and most deprecated functionality remained operational with warnings rather than failures. In contrast, maintaining older environments created significant barriers to utilizing modern hardware and libraries. In hindsight, beginning with a modern environment (e.g., Python 3.10 and recent PyTorch versions) and only downgrading when strictly necessary would have reduced overall development time and complexity.

Third, the importance of appropriate computational resources became evident. Initial development was conducted on a local machine with a single NVIDIA 3050 Ti GPU, which was sufficient for debugging and small-scale testing but inadequate for full-scale training. Transitioning to a HPC environment was unavoidable to complete training within a reasonable timeframe. However, this transition introduced additional overhead, including application for FIR access, job scheduling constraints, and the need to adapt scripts for distributed execution. In retrospect, integrating HPC resources from the outset would have streamlined

the development process and avoided mid-project disruptions.

Finally, the project highlighted the inherently uncertain nature of assembling large-scale machine learning systems. Constructing a Transformer from scratch requires integrating many interdependent components—architecture, data pre-processing, batching, optimization, and decoding—each of which may appear correct in isolation while still failing when combined. During most stages of development, there is no definitive signal that the overall system is functioning as intended. Meaningful evaluation metrics such as BLEU score are only obtainable once the entire pipeline is complete and trained, which occurs at the very end of the process. As a result, development often proceeded without confirmation, relying on the assumption that each component had been implemented correctly despite no immediate evidence that the full system would succeed.

This creates a setting in which progress must be made under persistent uncertainty. Intermediate checks—such as verifying tensor shapes, confirming gradient flow, or observing decreasing loss—provide only partial validation and cannot guarantee that the final objective will be achieved. It was not clear throughout much of the process whether the model would truly produce coherent translations or reach meaningful performance.

When translations finally began to resemble valid sentences, it served as the first concrete confirmation that the system was functioning end-to-end, and provided much needed relief. This experience underscores the importance of maintaining confidence in the implementation process despite the absence of continuous validation. Without that conviction, it becomes easy to hesitate or prematurely abandon directions that are in fact correct, ultimately stalling progress. In hindsight, successful replication required not only technical rigor, but also a willingness to continue forward in the absence of guarantees. If undertaken again, this process would be approached with greater confidence, recognizing that such uncertainty is intrinsic to building complex models and not a reason to fear alone.

7 Future Work

There multiple directions for future work that emerge from this study. Some involve infrastructure the project currently possesses while others

demand building further upon it.

First, it would be valuable to precisely replicate the token-based batches used in the paper rather than approximating via sentence counts. Given the sensitivity of training dynamics to batch composition, this could have a measurable impact on BLEU scores.

Second, exploring larger Transformer configurations would be an organic next step. The original paper reports that the *big* Transformer configuration achieves improvements of approximately 0.6 BLEU points over the base model when trained for up to 300,000 steps. Checking if the performance improvement is as staunch as the paper originally claimed serves to validate the original paper all the more.

Third, extending the model to additional language pairs would test the generality of the learned hyperparameters and preprocessing pipeline. In particular, languages with less shared subwords/grammar overlap, like english and russian, would possibly require different tokenization strategies, revealing deeper limitations or robustness of the original attention mechanism.

Fourth, a comparative study with contemporary RNN and CNN baselines would provide additional validation of the Transformer’s relative performance. Using the same BLEU evaluation pipeline constructed in this project would ensure consistency in comparison.

Finally, a more exploratory direction would be to adapt the architecture toward question answering or conversational tasks. Given that modern large language models are built upon Transformer variants, this would provide a bridge between historical architectures and contemporary applications.

References

- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. [Neural machine translation by jointly learning to align and translate](#). *Preprint*, arXiv:1409.0473.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, and 181 others. 2024. [Deepseek-v3 technical report](#). *Preprint*, arXiv:2412.19437.
- Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. 2017. [Convolutional sequence to sequence learning](#). *Preprint*, arXiv:1705.03122.
- Philipp Koehn, Hieu Hoang, Alexandra Birch, Chris Callison-Burch, Marcello Federico, Nicola Bertoldi, Brooke Cowan, Wade Shen, Christine Moran, Richard Zens, Chris Dyer, Ondřej Bojar, Alexandra Constantin, and Evan Herbst. 2007. [Moses: Open source toolkit for statistical machine translation](#). In *Proceedings of the 45th Annual Meeting of the Association for Computational Linguistics Companion Volume Proceedings of the Demo and Poster Sessions*, pages 177–180, Prague, Czech Republic. Association for Computational Linguistics.
- Philipp Koehn, Franz J. Och, and Daniel Marcu. 2003. [Statistical phrase-based translation](#). In *Proceedings of the 2003 Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics*, pages 127–133.
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, and 262 others. 2023. [Gpt-4 technical report](#). *Preprint*, arXiv:2303.08774.
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. [Bleu: a method for automatic evaluation of machine translation](#). In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA. Association for Computational Linguistics.
- Matt Post. 2018. [A call for clarity in reporting bleu scores](#). *Preprint*, arXiv:1804.08771.
- Alexander Rush. 2018. [The annotated transformer](#). In *Proceedings of Workshop for NLP Open Source Software (NLP-OSS)*, pages 52–60, Melbourne, Australia. Association for Computational Linguistics.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. [Neural machine translation of rare words with subword units](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.
- Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. [Sequence to sequence learning with neural networks](#). *Preprint*, arXiv:1409.3215.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). *Preprint*, arXiv:1706.03762.